GitHub Actions Performance Optimizer

A machine-learning, dependency-graph-aware study of CI/CD pipelines, evaluated on real workflow history



Progress review with Prof. Pushpa Lakshmi R, SCOPE

Akshat Singh (23BKT0005) Aditya Choubey (23BCE2274) BCSE497J, VIT Friday 16 October 2026

Every figure is labelled ESTIMATED or MEASURED; interim figures are as of 14 October 2026

Where the project stands

A working system, and an honest evaluation that is mostly negative on speed

1 Models beat a naive guess only in some repositories

Significant gain in fastapi and pydantic; never significantly worse elsewhere

2 Our improved model missed the bar we set in advance

2 of 9 repositories beaten; the pre-registered bar was 3

3 Scheduling is not faster by default

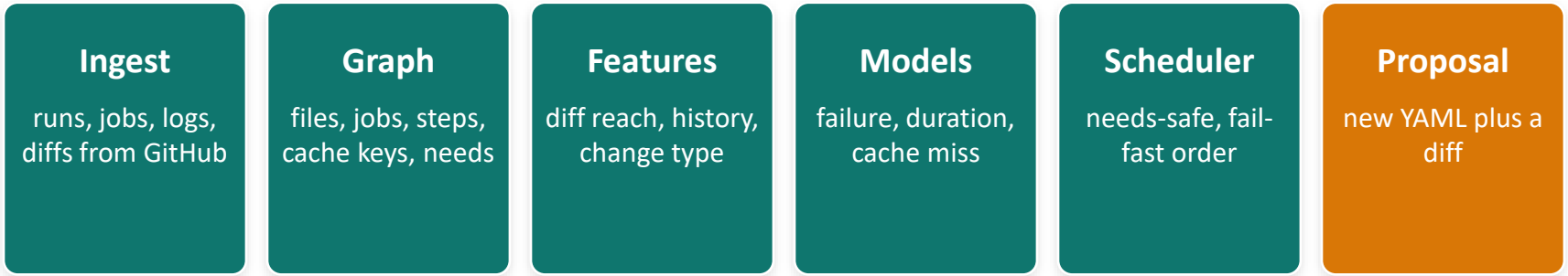
0% estimated; a measured -15% critical path on one simple pipeline

4 Real failures follow patterns

About half of failed jobs in poetry and fastapi are repeat alerts, not new problems

How the system works

From repository to a reviewable proposal, with real data only



Real data

Nine repositories; no synthetic values shown as results

Honest history

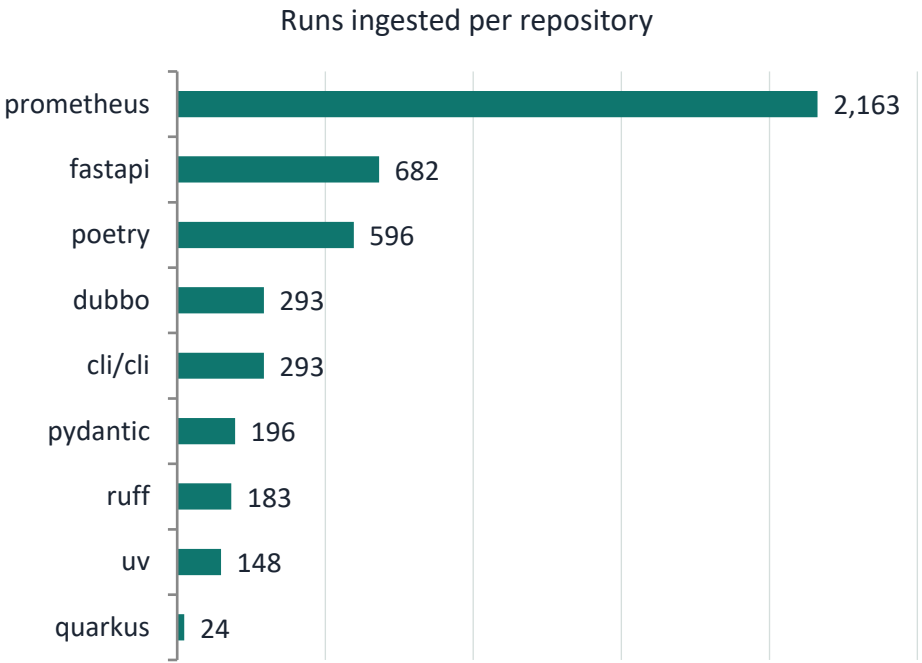
A run is visible to later runs only after it finished

Safe output

Original workflow never edited; needs rules kept

The evidence base: real CI history

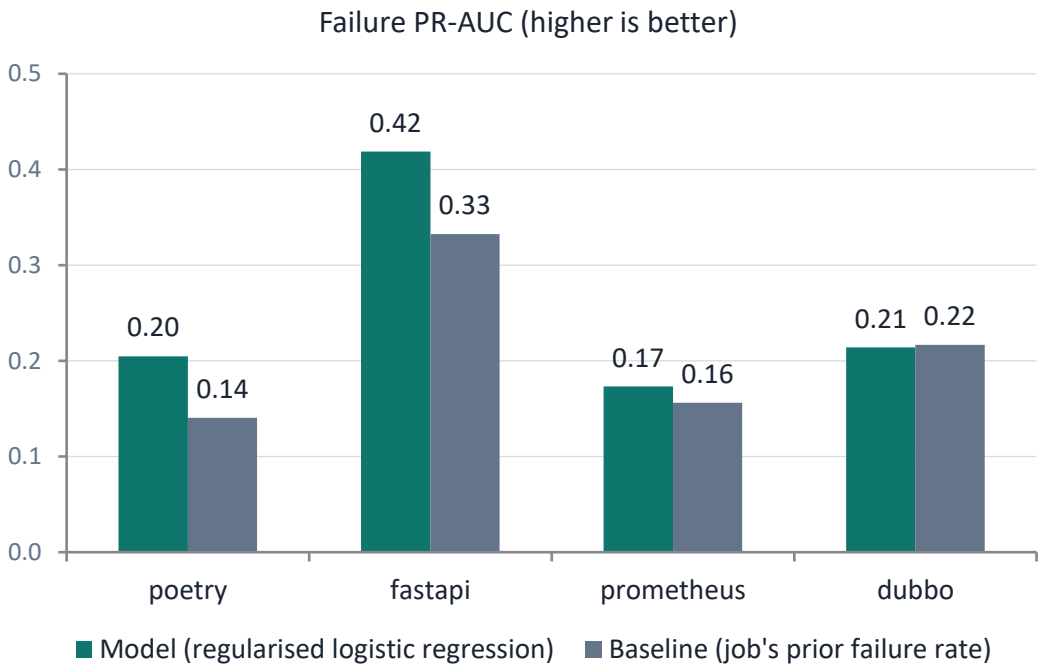
Nine repositories, about 4,600 runs, about 72,000 job rows



- Only information available when a run started is used: history comes from runs that had already finished
- Tests always run on later runs (rolling origin), never a random split
- Five repositories were never used to make a choice; plans were written first
- Results are always shown next to a naive baseline

Do the models beat a simple guess?

Only in some repositories, and never significantly worse



+0.085

fastapi: interval excludes 0

+0.110

pydantic (held out): interval excludes 0

poetry, prometheus, dubbo: not distinguishable from the baseline

The improvement attempt missed its bar

Pre-registered criterion not met; the pooled model is not holding up on fresh data

2 of 9

repositories beaten by the v2 model; the bar was 3 (not met)

3 of 9

pooled model on old data: exploratory, designed after seeing results

1 of 3

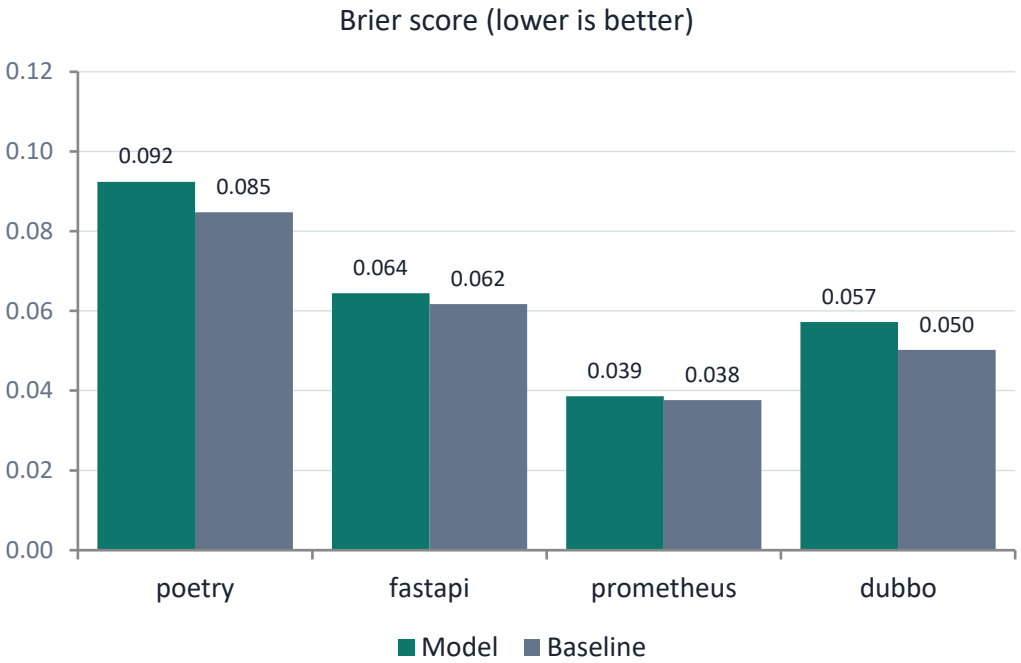
pooled model on fresh data so far: would be rejected if final

What this means

- We report the miss as a miss; the rule and the bar were not changed after seeing results
- Fresh-data verdicts are fixed for 21 October (logistic default) and 26 October (pooled model); today's numbers are interim

A failure score is not a probability

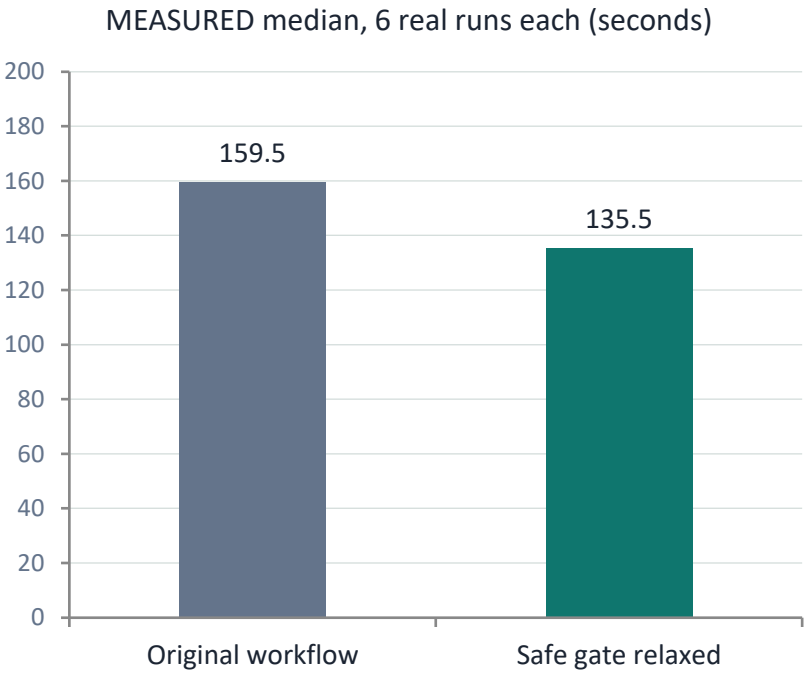
Recalibrating did not help, so the dashboard reports observed rates instead



- Top group over-confident: fastapi predicted 0.49, observed 0.37
- Platt scaling and an isotonic map did not improve the Brier score in 3 of 4 repositories
- poetry's main test job scored 0.91, above any score seen on unseen rows (highest 0.58)
- So the dashboard shows base rate and observed rate beside the score

Does scheduling make CI faster?

Measured on one simple pipeline; about zero on mature ones



-15.1%

critical path, p = 0.013

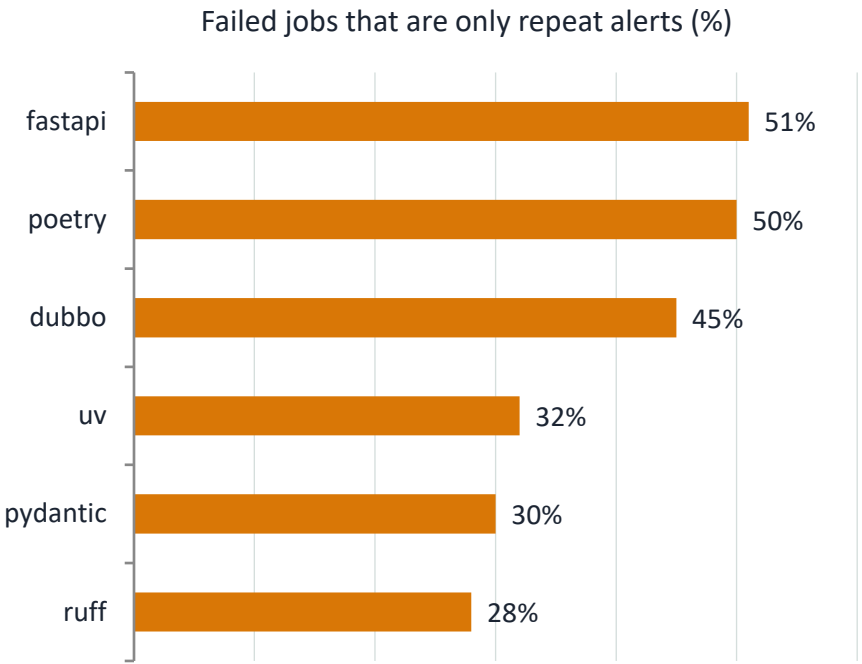
+4.1%

billable time, not significant

- Default plan: 0% estimated saving; replay on 9 repositories: about 0%
- Skipping unreached jobs saves 12-17% but hides 3-10% of failures: advisory only
- Generated workflow ran identically on real runs (+0.3%, p = 0.92)

What real CI failures look like

About half of failed jobs in poetry and fastapi only repeat an upstream failure



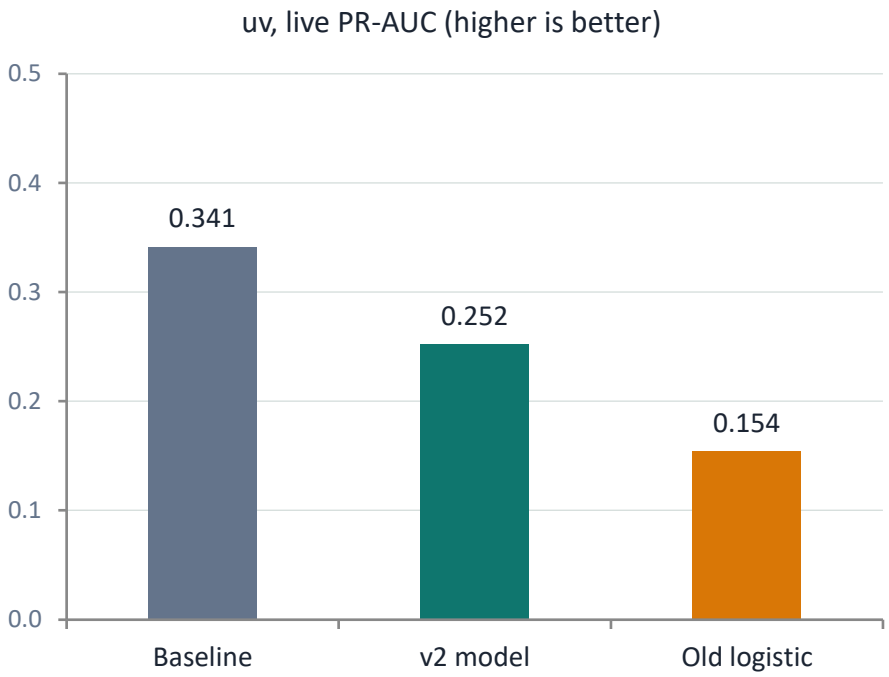
10-25%

of red runs in four repositories needed no code fix

- Classes: repeat alert, flaky, pre-existing, fixed later, persistent, unrelated
- Limits: flakiness under-counted; classes are heuristics, not checked against developers

Predicting live, before jobs finish

Interim, after two days: neither model beats the baseline on uv



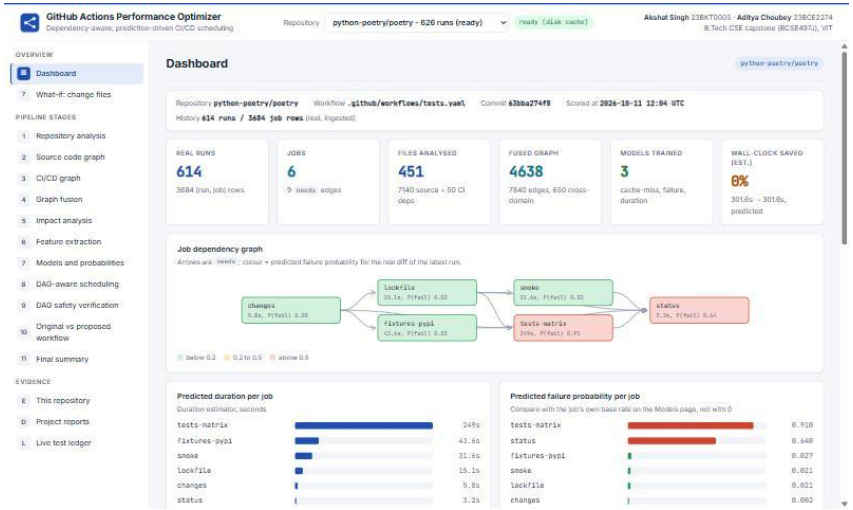
46
runs with outcomes

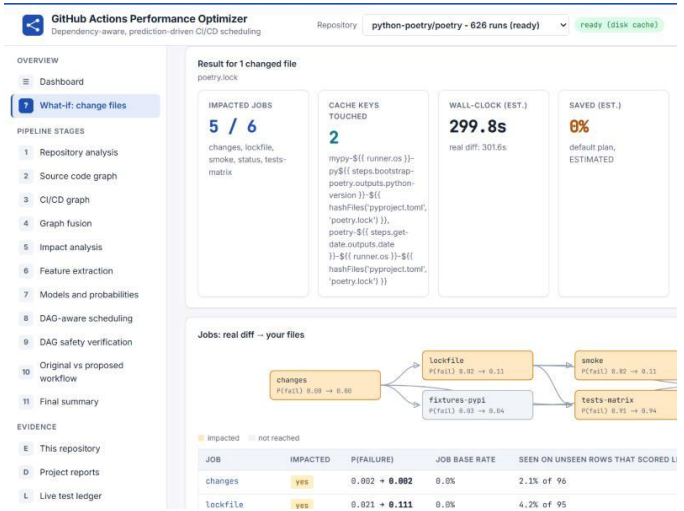
43
failing jobs of 739

- Old logistic vs baseline: -0.173 (excludes 0); v2: -0.082 (includes 0)
- ruff: no failures; prometheus: cannot be scored
- A demonstration, not a significance test

The dashboard

One page per stage, what-if scoring, and the evidence behind every claim





- Repository selector; a repository without enough history shows why it declined
- Score shown beside the job's base rate and the observed rate; original vs proposed workflow side by side

Engineering rigour

Same inputs, same numbers; the system refuses to guess

Reproducible

Scoring time is an explicit input; two full runs were byte-identical

428 tests

427 pass, 1 skipped; lint clean; fixtures are data, not results

Tamper-evident

Live predictions are hash-chained; editing one breaks the chain

Refuses to guess

Declines when real history is too thin: 3 of 9 repositories now

What is still open, and when

Final verdicts are fixed by the pre-registered rules

15 Oct	Automated refresh of the interim figures in this deck
19 Oct	Live test: final evaluation of the ledgers
21 Oct	Logistic default: fresh-data verdict (interim: 2 judged, both pass)
26 Oct	Pooled model: fresh-data verdict (interim: would be rejected)

Limits we state ourselves

- Nine repositories, 5-7 weeks of history each
- Few failures in several repositories: wide intervals
- Relaxed gates measured on one pipeline only
- Triage not checked against developer actions
- Live sample is small

What we claim, and what we do not

We claim

- A rigorous evaluation method with pre-registered tests
- Honest negative and mixed results, with data
- A working, tested pipeline that refuses to guess
- A failure triage from real outcomes

We do not claim

- A general speed-up of CI
- Calibrated failure probabilities
- That the pooled model works
- That triage advice matches developers

Guidance we would like

- Frame the write-up as an evaluation and negative-results study?
- Add more repositories now, or accept nine and state the limit?
- Which venue or report format should the final submission target?